



INSTITUTO TECNOLÓGICO DE BUENOS AIRES

Introducción a la Computación 22.06

Trabajo Práctico Final – Batalla Naval

Trabajo de Laboratorio N ° 7

Curso 2020A – 1er Cuatrimestre

GRUPO N°: 4		INTEGRANTES
Legajo N°	Nombre	
62490	Marcos Dario Alegre	
62012	Santiago Feldman	
62447	Anna Candela Gioia Perez	
61758	Juan Sebastián Helou	

	Fecha	Docente
Realizado	08/11/2020	
Presentado	11/11/2020	
Aprobado		



Contenido:

Consigna, metodología del juego y herramientas de programación	Página N°3
Diagramas de flujo	Página N°6
Presentación del código del programa	Página N°11



Trabajo práctico Especial

Trabajo Práctico Final

Consigna:

Se pide desarrollar una batalla naval de dos jugadores utilizando la versión especial del simulador de wookie, Wookie FX podrán encontrar dicha versión en el iol /ik.

Metodología de Juego:

Se definirán 2 tableros de 6 x 6 posiciones en los cuales se distribuirán 5 embarcaciones: 2 cargueros y 3 submarinos. Los cargueros ocupan 2 posiciones en el tablero, mientras que los submarinos solamente una¹. La disposición de los barcos se hará respetando que los mismos no se encuentren contiguos entre sí o que se superpongan (un submarino no puede estar en el mismo casillero que un carguero). La distribución de las embarcaciones se hará previa a la compilación en 2 matrices de 6 x 6 bytes.

En ningún momento de la partida se mostrará la distribución de los barcos de cualquier jugador. Comienza el juego, el **jugador 1** elegirá una posición a disparar y presionará el botón de disparo. En el display se mostrará el resultado de dicho disparo: agua, tocado o hundido. El **jugador 2** presionará el botón P2 para indicar que comienza su turno. Una vez mostrada su pantalla de disparo el **jugador 2** seleccionará el casillero a disparar, luego de presionar el botón de disparo se mostrará nuevamente el resultado de su intento. El **jugador 1** para retomar su turno presionará el botón P1.

El juego concluye cuando cualquier jugador hunde toda la flota del contrario. Se indicará la victoria con un mensaje en el display.

Caso de uso: si un jugador ya tiene el turno y vuelve a apretar Player 1 o Player 2, se repite la operación de selección de jugador, lo que no tiene otras consecuencias que “borrar” una entrada parcial de coordenadas antes de apretar “disparo.”

Herramientas de Programación:

Cada posición/celda del display está asociada a una posición de memoria que comienza en \$5F00 (en nuestro caso: matriz de 6x6 el rango de memoria ocupado va de \$5F00 a \$5F23).

Cada celda ocupa 1 byte. Dicho byte indica el color de la celda y en nuestro caso el tipo de embarcación. La posición de memoria anterior a la matriz (\$5EFF se utiliza para

¹ Revisar condiciones de entrega respecto de las embarcaciones solicitadas



indicar el modo de lectura de los colores, si la misma contiene un 0 los colores se leerán según RGB² y tuviese otro valor la descripción de colores será GRB).

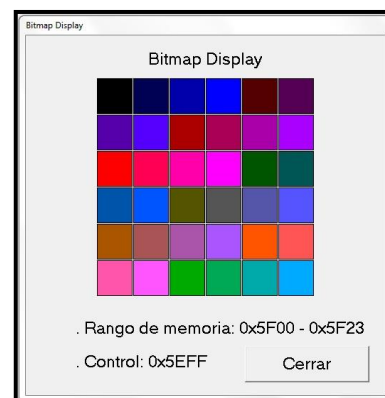
Cada celda tendrá una distribución de bits acorde a la tabla siguiente³:

Celda de memoria típica							
b7	b6	b5	b4	b3	b2	b1	b0
tb	tb	rr	rr	gg	gg	bb	bb
Tipo de Barco		Red		Green		Blue	

Como convención de la cátedra estipulamos los siguientes valores para describir los siguientes estados⁴:

Configuración de Bits				
Tipo de Embarcación	Imagen	b7	b6	Posiciones
Sin Embarcación (agua)		0	0	
Submarino		0	1	1
Carguero		1	0	2

Estado	Color
Agua	Azul
Tocado	Amarillo
Hundido	Rojo
Disparado	Verde



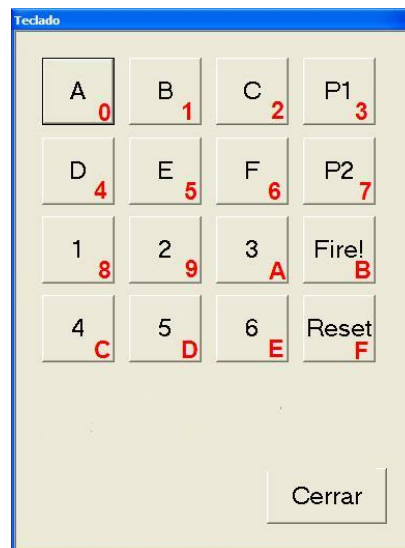
² RGB: Red Green Blue

³ Asumiendo que se elije la distribución RGB en la posición de memoria \$5EFF

⁴ Tener en cuenta que el Bitmap del display en general está inicializado con valores aleatorios.



Por último se define la siguiente secuencia de teclas para jugar:



Solamente es requisito para aprobar el TP que la batalla naval funcione con 6 embarcaciones (pueden ser todas submarinos). Agregar cargueros es opcional y contribuirá a una mejor calificación.

Importante:

- Documentar claramente el programa.
- Armar un esqueleto para el programa. Definir funciones y un diagrama de flujo antes de empezar a programar. Modularizar el desarrollo al máximo.
- Utilizar el stack en forma eficiente.
- Es recomendable revisar la inicialización del display
- Utilizar constantes simbólicas para clarificar el código (colores, barcos, etc)
- Seguir los criterios de buena programación.
- Se premiarán diseños creativos que utilicen la mayor parte de los conceptos dados en la materia así como también aquellos diseños que mejoren la experiencia del usuario.

Fecha de entrega: hasta el 11 de noviembre de 2020 inclusive (no se admiten entregas posteriores si anteriores, a partir del 4 de noviembre de 2020 se habilita una fecha de entrega parcial del TP para los que la requieran

La entrega consta de 2 partes. Para considerarse aprobado el TP se deberá presentar la Batalla Naval funcionando en una máquina de la universidad y luego presentar el código. En la presentación el docente evaluará la participación de cada miembro en el desarrollo del trabajo práctico así como el funcionamiento general del proyecto.



Diagramas de flujo:

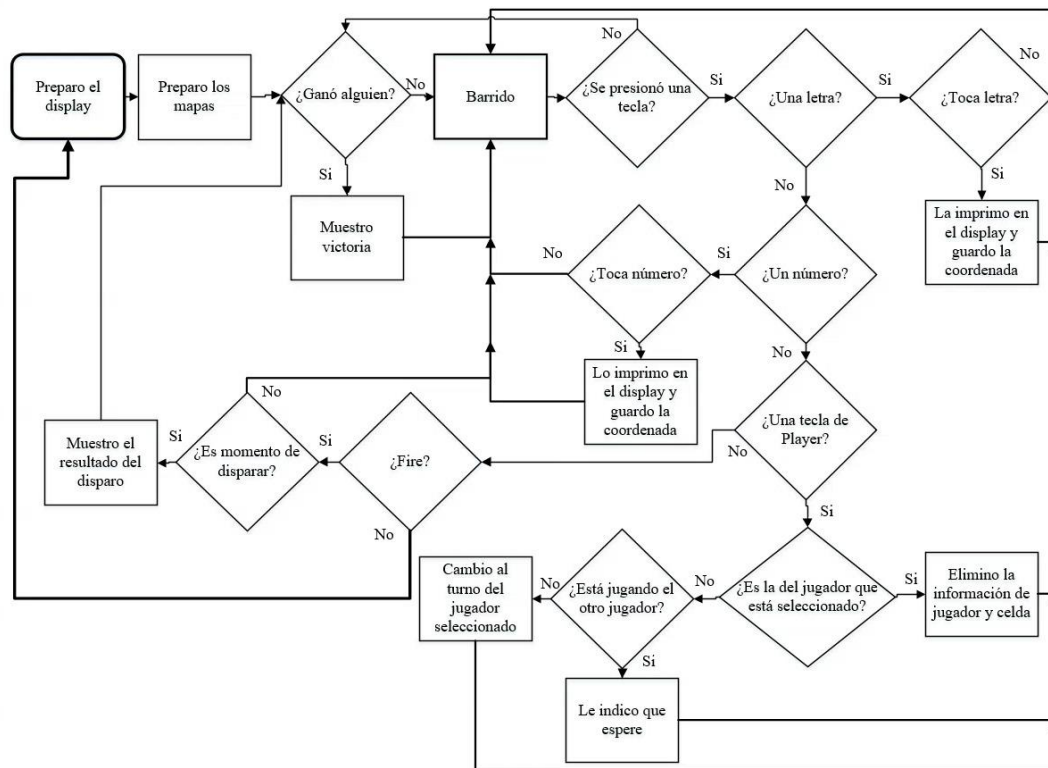


Diagrama 1.1: diagrama de flujo de TBN

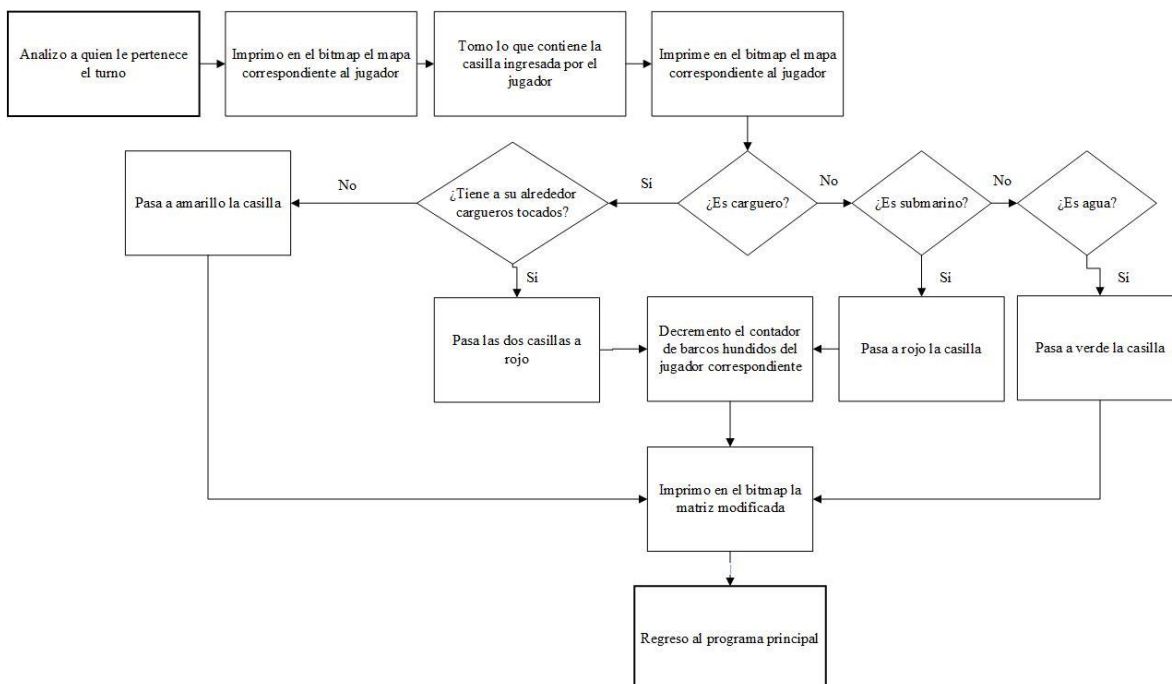


Diagrama 1.2: diagrama de FIREC

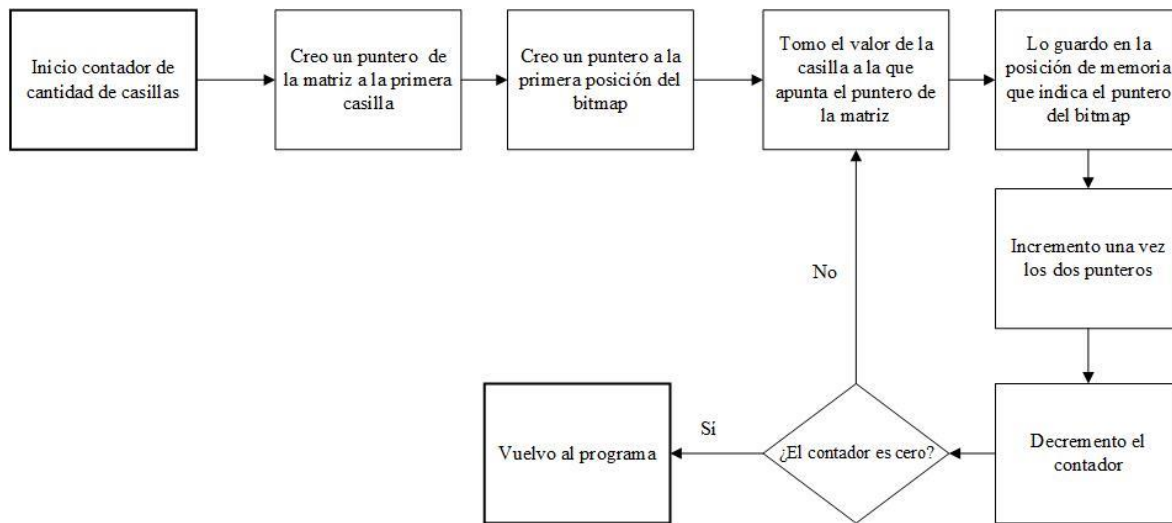


Diagrama 1.3: diagrama que explica la impresión de la matriz al bitmap

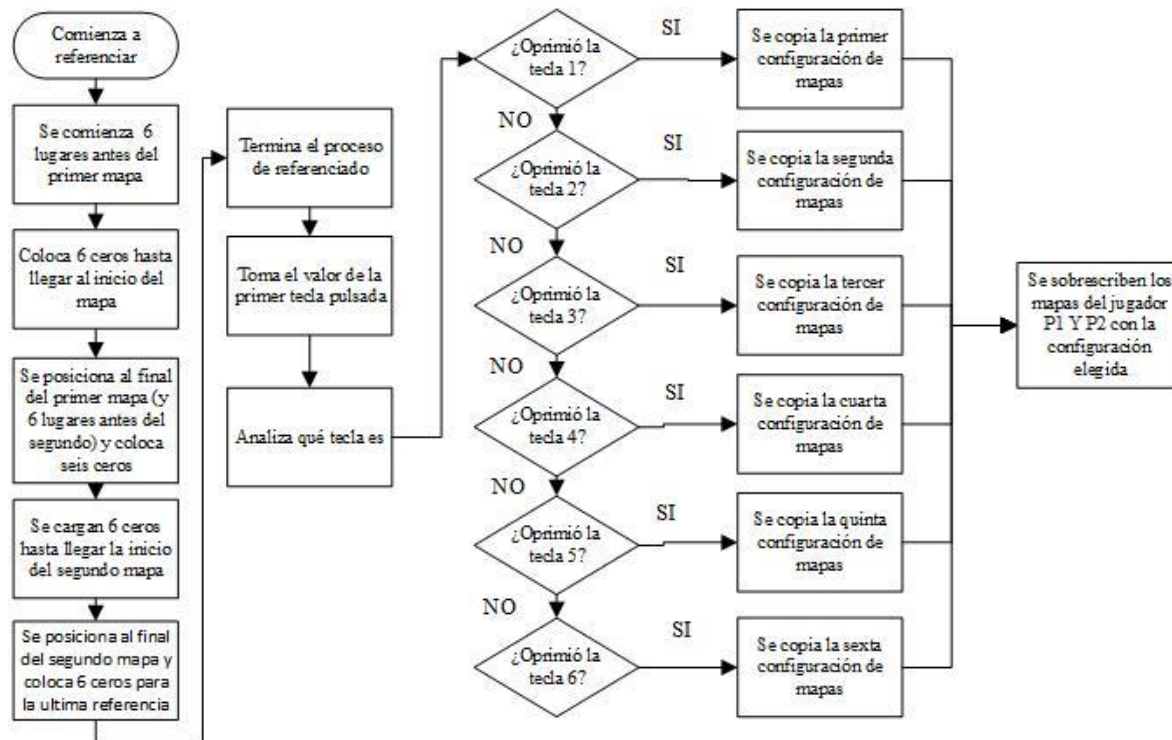


Diagrama 1.4: diagrama de selección de mapas

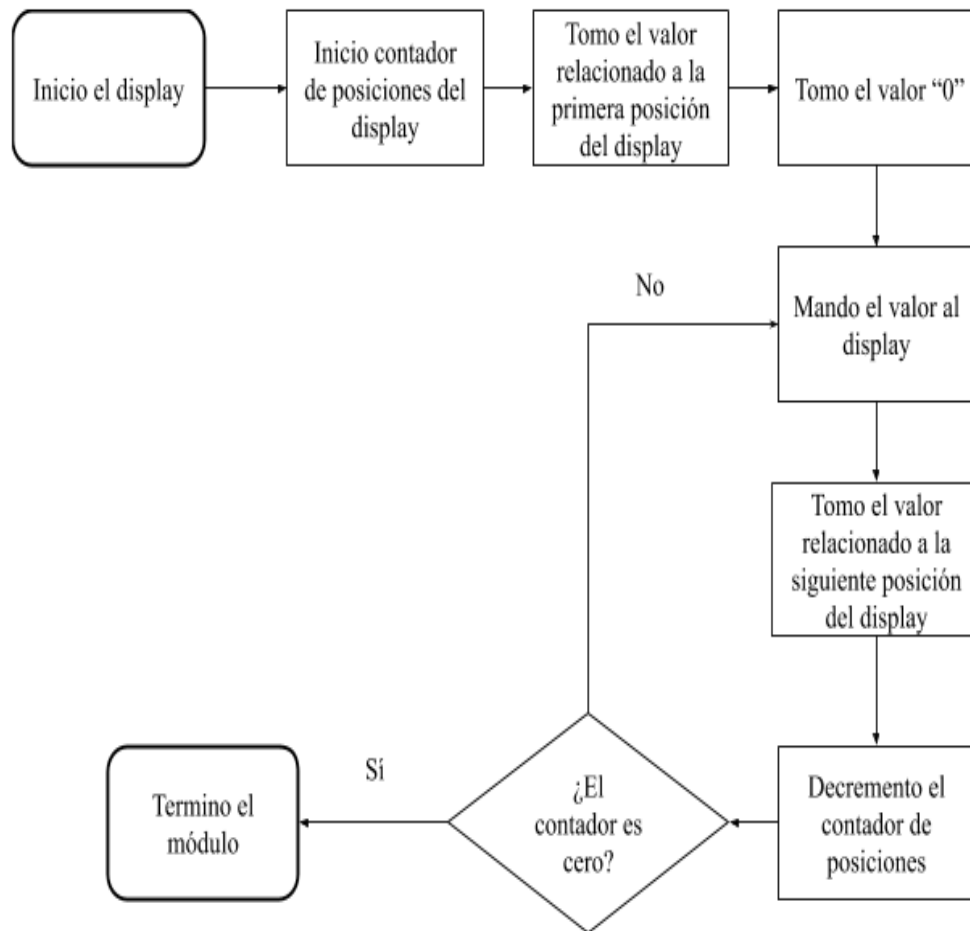


Diagrama 1.5: diagrama de flujo del borrado del display

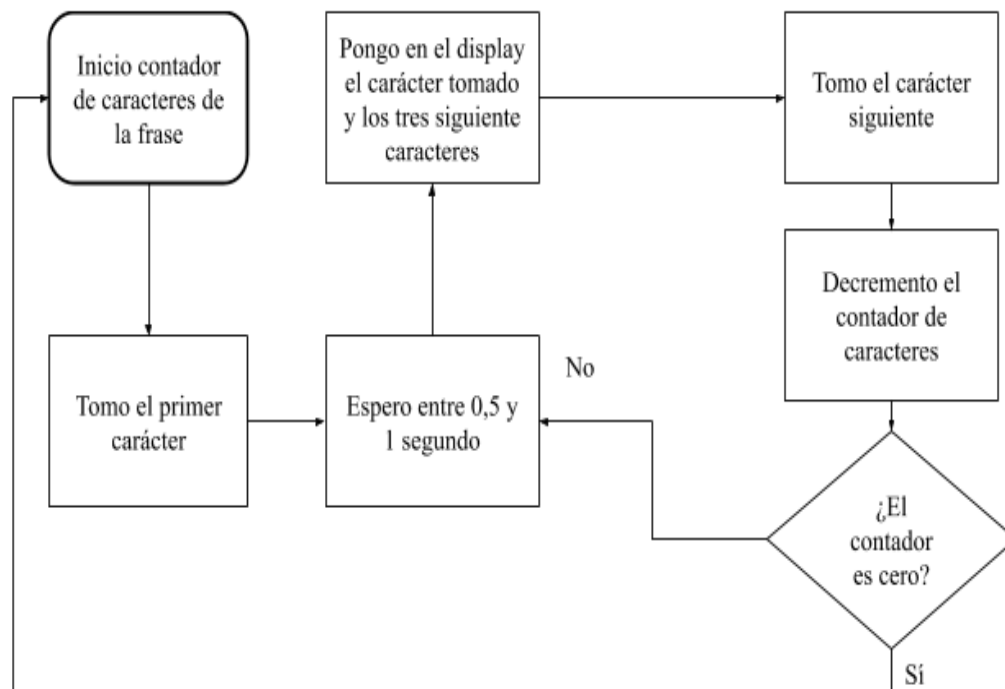


Diagrama 1.6: diagrama del display; explica cómo se exhiben caracteres

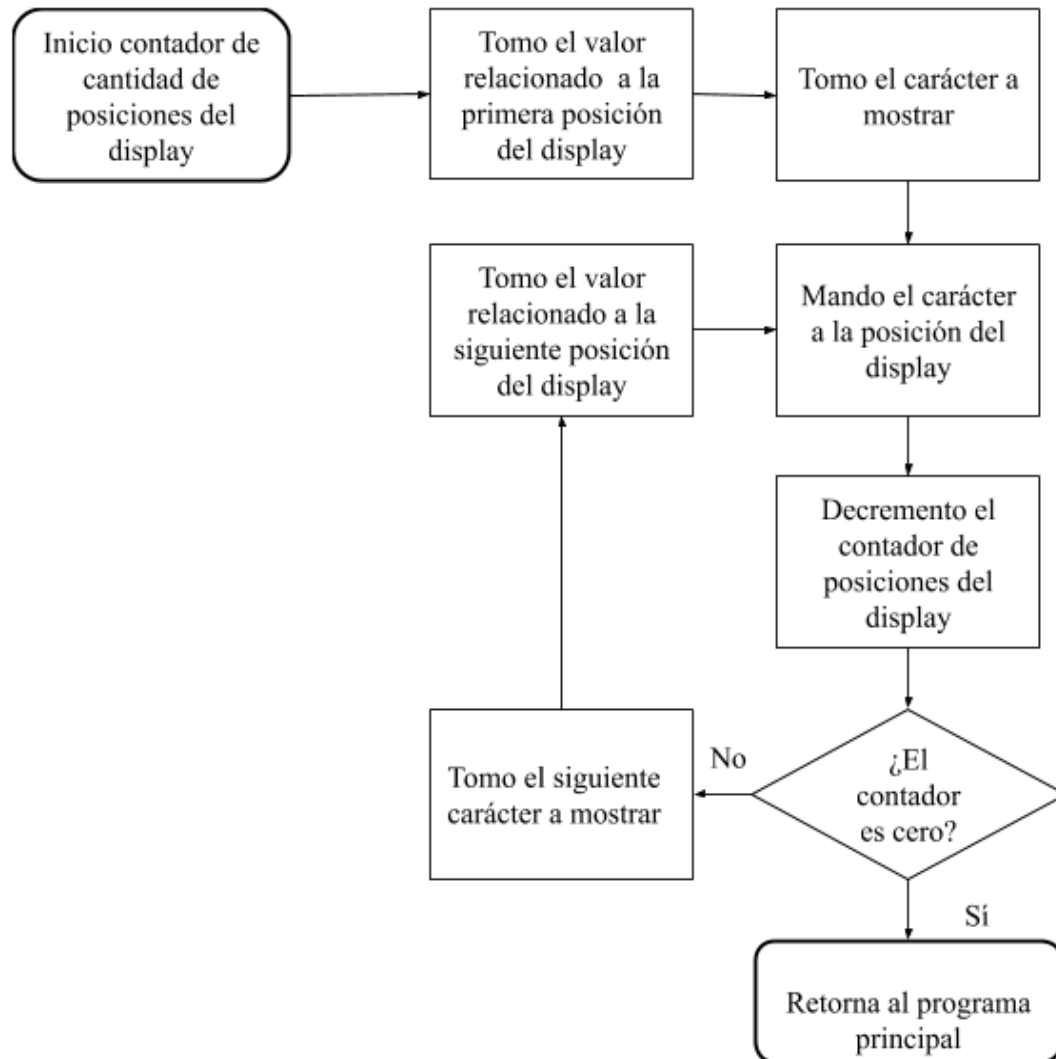


Diagrama 1.7: diagrama de flujo del display; explica el movimiento de derecha a izquierda de los caracteres



Presentación del código del programa:

TBN.LST

```
#####
#
#   Micro Series 6801 Assembler V2.00/DOS           10/Nov/20   22:47:31   #
#
#   Source   =   tbn.msa                               #
#   List     =   tbn.lst                               #
#   Object   =   tbn.r07                               #
#   Options  =                                         #
#
#                                           (c) Copyright IAR Systems 1990 #
#####

1 0000                                p68h11
2
5
6 0000                                extern  dofire,loop1,wins,ran
7 0000                                public  flagp,winp1,winp2,flagmap,MAP1,MAP2,MAP3,MAP4,MAP5,MAP6,MAP7,AGUA,BARC,CARG
8
9                                *****
10                               *                AREA DE EQUs                *
11                               *                *
12                               *   Se definen labels que se utilizaran   *
13                               *                en el programa                *
14                               *                *
15                               *****
16
17                                $display3.def
18                                *****Ports*****
19
20 1007                                DDRC                equ      $1007                ;data direction register for C
21 1002                                PIOC                equ      $1002                ;for strobe B assertion change
22 1005                                PORTCL              equ      $1005                ;port C latched
23 1004                                PORTB               equ      $1004                ;port B
24 1003                                PORTC               EQU      $1003                ;rows
25
26                                *****Cols and Row Ports*****
27
28 1003                                ROWS                EQU      PORTC                ;outputs
29 1005                                COLS                EQU      PORTCL              ;inputs
30
31                                *****Row Masks*****
32
33 000E                                FIL0                EQU      00001110b            ;mascaras para manejo de filas
34 000D                                FIL1                EQU      00001101b
35 000B                                FIL2                EQU      00001011b
36 0007                                FIL3                EQU      00000111b
37
38                                *****Keyboard Mask*****
39
40 00EE                                MASKA                EQU      11101110b
41 00DE                                MASKB                EQU      11011110b
42 00BE                                MASKC                EQU      10111110b
43 00ED                                MASKD                EQU      11101101b
44 00DD                                MASKE                EQU      11011101b
45 00BD                                MASKF                EQU      10111101b
46
```



```
47 00EB      MASK1      EQU      11101011b
48 00DB      MASK2      EQU      11011011b
49 00BB      MASK3      EQU      10111011b
50 00E7      MASK4      EQU      11100111b
51 00D7      MASK5      EQU      11010111b
52 00B7      MASK6      EQU      10110111b
53
54 007E      MASKP1     EQU      01111110b
55 007D      MASKP2     EQU      01111101b
56 007B      MASKFR     EQU      01111011b
57 0077      MASKR      EQU      01110111b
58
59 *****Display*****
60
61 0003      DI_START    EQU      $03
62 0000      DI_END      EQU      $00
63
64
65 0009      L           EQU      $9
66 00FF      M           EQU      $FF
67 00FF      S           EQU      $FF
68
69
70 0003      AGUA        EQU      00000011b      ;valor del color azul y agua
71 0043      BARC        EQU      01000011b      ;valor del submarino
72 0083      CARG        EQU      10000011b      ;valor del carguero
73
74
75
76 *****
77 *                AREA DE MACROS                *
78 *                *                                *
79 *      Aqui se definen las macros que            *
80 *      seran utilizadas a lo largo               *
81 *      del programa                               *
82 *                *                                *
83 *****
84
85 0000      MACRO      %LETRA
86
87          ldaa      flagmsel      ;se fija si ya se eligieron mapas
88          beq      setmap\@      ;si no se eligieron lo elige
89          ldaa      #\0          ;si
90          jsr      chklet      ;se fija se toca seleccionar letra
91          jsr      muestro      ;si toca la muestra en el display
92          ldaa      #\1          ;y guarda en el stack la coordenada
93          psha
94          jmp      barrido      ;vuelve a barrer
95
96      setmap\<      inc      flagmsel
97                  ldaa      #\2
98                  staa      flagmap      ;guarda el mapa seleccionado
99                  ldab      #$30
100                 aba
101                 ldab      #DI_END
102                 jsr      outchar
103                 jsr      ran      ;lo imprime en memoria
104                 jmp      barrido      ;vuelve a barrer
105
106 0000      ENDMAC
107
```



```
108
109 0000          MACRO  %NUMERO
110
111              ldab    counter
112              cmpb    #1          ;se fija si toca seleccionar numero
113              bne     \0          ;si no toca sigue barriendo
114              ldaa    #\1          ;si toca guarda la coordenada en el stack
115              psha
116              ldaa    #\2          ;y muestra el numero en el display
117              jsr     muestro
118              jmp     barrido      ;vuelve a barrer
119
120 0000          ENDMAC
121
122
123
124 *****
125 *              MAIN PROGRAM      *
126 *
127 * Este programa tiene como objetivo *
128 * replicar el funcionamiento del   *
129 * juego de mesa BATALLA NAVAL     *
130 *
131 *****
132
133 0000          rseg    CODE
134
135 0000 BD0000    jsr     initdis      ;se inicializa el display
136
137 0003 8E0063    Reset      lds     #stackP
138 0006 8606      ldaa     #6
139 0008 B70008    staa     winp1      ;se reinician los contadores de victoria
140 000B B70009    staa     winp2
141 000E 7F000A    clr      flagmsel   ;se reinicia el indicador de seleccion de mapa
142
143 0011 BD00E2    jsr     selec
144
145 0014 C603      vas        ldab    #DI_START      ;toma el valor de la primera posición del display
146 0016 CE0004    ldx      #4          ;inicia contador de posiciones del display
147 0019 8600      ldaa     #0          ;toma el valor 0 para borrar el display
148 001B BD0010    cleandisp jsr     outchar        ;imprime
149 001E 5A        decb     ;decrementa la posición del display
150 001F 09        dex      ;decrementa el contador de posiciones
151 0020 26F9      bne      cleandisp   ;si el contador no es 0 repite la secuencia
152
153 0022 8600      ldaa     #0          ;se inicia el flag de jugador en 0 para que cualquiera
154 0024 B70007    staa     flagp      ;pueda jugar primero
155
156 0027 C602      primedisp  ldab    #2
157 0029 F70000    stab     counter     ;se preparan las posiciones de memoria que funcionan
158 002C 8600      ldaa     #0          ;como contador y como retenedor de caracteres
159 002E B70003    staa     msg+1
160
161
```



```
162
163 *****
164 *          SECCION DE TECLADO          *
165 *                                     *
166 * Esta parte del programa se encarga *
167 * de leer constantemente los valores *
168 * del Puerto C para fijarse si alguna *
169 * tecla es presionada, registrar esa *
170 * tecla y realizar las acciones      *
171 * necesarias para el desarrollo      *
172 * de la partida                      *
173 *                                     *
174 *****
175
176 0031 B60008   barrido      ldaa    winp1
177 0034 2707          beq      vic          ;se fija si gano algun jugador
178 0036 B60009   ldaa    winp2          ;y si es asi lo muestra
179 0039 2702          beq      vic
180 003B 2006          bra      barrido0
181
182 003D BD00A6   vic          jsr      victoria
183 0040 BD0000          jsr      wins
184
185 0043 BD0017   barrido0     jsr      delay          ;se genera un poco de retraso en el barrido para
186 *****;evitar presionar involuntariamente la misma tecla
187 *****;mas de una vez
188 0046 C60E          ldab    #FIL0
189 0048 F71003          stab    PORTC          ;se prepara la mascara de la fila 0
190
191 *****;SE PRESIONA EL BOTON
192
193 004B B61003          ldaa    PORTC
194 004E 81EE          cmpa    #MASKA
195 0050 270F          beq      writeA
196 0052 81DE          cmpa    #MASKB
197 0054 2734          beq      writeB          ;se compara el valor del puerto c con los valores
198 0056 81BE          cmpa    #MASKC          ;correspondientes a cada tecla de la fila 0 para
199 0058 2759          beq      writeC          ;detectar si se presiono una tecla
200 005A 817E          cmpa    #MASKP1
201 005C 277E          beq      writeP1
202 005E 7E00DF          jmp      barrido1
203
204
205 0061          writeA      %LETRA $11,0,1
206
207 008A          writeB      %LETRA $12,1,2
208
209 00B3          writeC      %LETRA $13,2,3
210
211 00DC 7E024F   writeP1     jmp      P1
212
213
214 00DF C60D   barrido1     ldab    #FIL1
215 00E1 F71003   stab    PORTC          ;se prepara la mascara de la fila 1
216
217 *****;SE PRESIONA EL BOTON
218
219 00E4 B61003          ldaa    PORTC
220 00E7 81ED          cmpa    #MASKD
221 00E9 2712          beq      writeD
222 00EB 81DD          cmpa    #MASKE
```



```
223 00ED 2737      beq      writeE      ;se compara el valor del puerto c con los valores
224 00EF 81BD      cmpa     #MASKF      ;correspondientes a cada tecla de la fila 1 para
225 00F1 275C      beq      writeF      ;detectar si se presiono una tecla
226 00F3 817D      cmpa     #MASKP2
227 00F5 2703      beq      writeP2
228 00F7 7E0184    jmp      barrido2
229
230 00FA 7E0264    writeP2      jmp      P2
231
232 00FD           writeD      %LETRA $14,3,4
233
234 0126           writeE      %LETRA $15,4,5
235
236 014F           writeF      %LETRA $16,5,6
237
238
239 0178 F60000    chklet      ldab      counter
240 017B C102      cmpb     #2          ;se fija si toca seleccionar letra
241 017D 2601      bne      notocal      ;si no toca vuelve a barrer
242 017F 39        rts
243
244 0180 32        notocal      pula
245 0181 32        pula
246 0182 2000      bra       barrido2      ;vuelve a barrer
247
248
249
250
251
252 0184 C60B      barrido2     ldab      #FIL2
253 0186 F71003    stab      PORTC          ;se prepara la mascara de la fila 2
254
255 *****;SE PRESIONA EL BOTON
256
257 0189 B61003    ldaa      PORTC
258 018C 81EB      cmpa     #MASK1
259 018E 270F      beq      write1
260 0190 81DB      cmpa     #MASK2
261 0192 271D      beq      write2      ;se compara el valor del puerto c con los valores
262 0194 81BB      cmpa     #MASK3      ;correspondientes a cada tecla de la fila 2 para
263 0196 272B      beq      write3      ;detectar si se presiono una tecla
264 0198 817B      cmpa     #MASKFR
265 019A 2739      beq      writeFire
266 019C 7E01E0    jmp      barrido3
267
268
269 019F           write1      %NUMERO barrido3,0,1
270
271 01B1           write2      %NUMERO barrido3,1,2
272
273 01C3           write3      %NUMERO barrido3,2,3
274
275 01D5 B60000    writeFire    ldaa      counter      ;si no toca disparar no dispara
276 01D8 2606      bne      barrido3
277 01DA BD0000    jsr      dofire
278 01DD 7E0027    jmp      primedisp      ;se prepara para recibir otra coordenada
279
280
281 01E0 C607      barrido3     ldab      #FIL3
282 01E2 F71003    stab      PORTC          ;se prepara la mascara de la fila 3
283
```



```
284 *****;SE PRESIONA EL BOTON
285
286 01E5 B61003      ldaa    PORTC
287 01E8 81E7        cmpa    #MASK4
288 01EA 270F        beq     write4
289 01EC 81D7        cmpa    #MASK5
290 01EE 271D        beq     write5      ;se compara el valor del puerto c con los valores
291 01F0 81B7        cmpa    #MASK6      ;correspondientes a cada tecla de la fila 3 para
292 01F2 272B        beq     write6      ;detectar si se presiono una tecla
293 01F4 8177        cmpa    #MASKR
294 01F6 2739        beq     writeReset
295 01F8 7E0031      jmp     barrido
296
297
298 01FB              write4      %NUMERO barremos,3,4
299
300 020D              write5      %NUMERO barremos,4,5
301
302 021F              write6      %NUMERO barremos,5,6
303
304 0231 8606        writeReset  ldaa    #6
305 0233 B70008        staa    winp1      ;reinicia los counters de condicion de victoria
306 0236 B70009        staa    winp2
307 0239 7E0003      jmp     Reset      ;reinicia
308
309
310 023C 7E0031      barremos    jmp     barrido      ;vuelve a barrer
311
312
313 023F C101        nuestro    cmpb    #1
314 0241 2705        beq     seg      ;si es el numero lo guarda en la segunda celda
315 0243 B70005      staa    celda1      ;si es la letra la guarda en la primera celda
316 0246 2003        bra     mues
317 0248 B70006      seg        staa    celda2
318 024B BD0031      mues        jsr     show      ;muestra lo seleccionado hasta el momento
319 024E 39          rts
320
321
322 024F B60007      P1          ldaa    flagp
323 0252 8101        cmpa    #1      ;si todavia esta jugando el P1 vuelve a barrer
324 0254 2723        beq     yej
325 0256 B60000      ldaa    counter
326 0259 8102        cmpa    #2      ;si esta jugando el P2 indica a traves del display que
327 025B 2642        bne     noturno    ;no es su turno
328 025D 8601        ldaa    #1
329 025F B70007      staa    flagp      ;si le toca al P1 se indica que esta jugando
330 0262 2027        bra     prntpl     ;y se muestra en el display
331
332
333 0264 B60007      P2          ldaa    flagp
334 0267 8102        cmpa    #2      ;si todavia esta jugando el P2 vuelve a barrer
335 0269 270E        beq     yej
336 026B B60000      ldaa    counter
337 026E 8102        cmpa    #2      ;si esta jugando el P1 indica a traves del display que
338 0270 262D        bne     noturno    ;no es su turno
339 0272 8602        ldaa    #2
340 0274 B70007      staa    flagp      ;si le toca al P2 se indica que esta jugando
341 0277 2012        bra     prntpl     ;y se muestra en el display
342
343 0279 B60000      yej        ldaa    counter
344 027C 2708        beq     ye2
```




```
345 027E 8101          cmpa    #1          ;Caso de uso: si un jugador ya tiene el turno y vuelve
346 0280 2606          bne     yen          ;a apretar Player 1 o Player 2, se repite la operación
347 0282 32            pula          ;de selección de jugador, lo que no tiene otras
348 0283 7E0014        jmp     vas          ;consecuencias que "borrar" una entrada parcial de
349 0286 32            pula          ;coordenadas antes de apretar "disparo.
350 0287 32            pula
351 0288 7E0014        yen          jmp     vas
352
353
354 028B C630          prntpl        ldab    #$30
355 028D 1B            aba
356 028E C600          ldab    #DI_END
357 0290 BD0010        jsr     outchar      ;se imprime el numero de jugador
358 0293 5C            incb
359 0294 8650          ldaa    #$50
360 0296 BD0010        jsr     outchar      ;se imprime la P
361 0299 BD0000        jsr     loop1
362 029C 7E0027        jmp     primedisp    ;reinicia el counter, prepara msg y vuelve a barrer
363
364
365 029F BD006A        noturno        jsr     espere          ;si no es el turno, lo indica en el display
366 02A2 B60000        ldaa    counter      ;se fija si ya había seleccionado el numero
367 02A5 2708          beq     nt2num        ;si es así muestra los dos caracteres
368 02A7 7C0000        inc     counter
369 02AA 8600          ldaa    #0          ;si no muestra solo la letra
370 02AC B70006        staa    celda2
371 02AF BD0031        nt2num        jsr     show
372 02B2 7E0031        jmp     barrido      ;vuelve a barrer
373
374
375
376 *****
377 *          SECCION DE SUBROUTINAS          *
378 *                                          *
379 *  Aquí se encuantran las subrutinas      *
380 *  de delay, de inicializacion del        *
381 *  display y de envio de caracteres al    *
382 *  display                                *
383 *                                          *
384 *****
385
386 *****
387 *          SUBROUTINA INITDIS              *
388 *                                          *
389 *          Inicia el display                *
390 *                                          *
391 *****
392
393 0000                rseg     SUBR
394
395 0000          initdis    EQU     *
396
397 0000 36            psha
398
399 0001 860F          ldaa    #$0F
400 0003 B71007        staa    DDRC
401 0006 B61002        ldaa    PIOC          ;se prepara el display
402 0009 84FE          anda    #$FE
403 000B B71002        staa    PIOC
404
405 000E 32            pula
```



```
406 000F 39          rts
407
408
409
410 *****
411 *          SUBROUTINA OUTCHAR          *
412 *                                     *
413 *          Envia caracteres al display  *
414 *                                     *
415 *****
416
417 0010      outchar      rseg      SUBR
418
419 0010 F71005          stab      PORTCL      ;se envian los caracteres
420 0013 B71004          staa      PORTB
421
422 0016 39          rts
423
424
425
426 *****
427 *          SUBROUTINA DELAY          *
428 *                                     *
429 *          Produce una demora        *
430 *                                     *
431 *****
432
433 0017      delay      rseg      SUBR
434
435 0017          org      *
436 0017 36          psha
437 0018 37          pshb      ;se hace un backup de los registros a modificar
438 0019 3C          pshx
439
440 001A CE0009          ldx      #L          ;crea el contador mayor
441 001D 09      countl      dex
442 001E 270D          beq      endelay      ;si es cero el contador mayor va a terminar el programa
443
444 0020 86FF          ldaa      #M          ;si no es cero reinicia el contador mediano
445 0022 4A      countm      deca
446 0023 27F8          beq      countl      ;si es cero el contador mediano, decrementará el mayor
447
448 0025 C6FF          ldab      #S          ;si no es cero reinicia el contador menor
449 0027 5A      counts      decb
450 0028 26FD          bne      counts
451 002A 7E0022          jmp      countm      ;si es cero el contador menor decrementa el mediano
452
453 002D 38      endelay      pulx
454 002E 33          pulb      ;se recuperan los valores guardados
455 002F 32          pula
456 0030 39          rts
457
458
```



```
459
460 *****
461 *          SUBROUTINA SHOW          *
462 *          *                         *
463 *      Acomoda los caracteres para ser *
464 *      enviados al display           *
465 *          *                         *
466 *****
467
468 0031                      rseg      SUBR
469
470 0031 C600      show      ldab      #DI_END      ;toma el valor de la primera posición del display
471 0033 CE0004      ld      #4      ;inicia contador de posiciones del display
472 0036 8600      ldaa      #0      ;toma el valor 0 para borrar el display
473 0038 BD0010      jsr      outchar      ;llama a la subrutina para poner los valores del
display
474 003B 5C                      incb      ;decrementa la posición del display
475 003C 09                      dex      ;decrementa el contador de posiciones
476 003D 26F9      bne      cleanshow      ;si el contador no es 0 repite la secuencia
477
478 003F 18CE0002      ldy      #msg      ;se apunta IY a donde se guardaran los caracteres
479
480
481 0043 B60005      ldaa      celda1      ;se carga a con el ultimo digito tecleado
482 0046 C630      ldab      #$30      ;se lo convierte a ASCII
483 0048 1B                      aba
484 0049 18A700      staa      0,y      ;se retiene al caracter en una posicion de memoria
485 004C 7A0000      dec      counter      ;se actualiza la cantidad de digitos ingresados
486 004F 2609      bne      undigit      ;si el numero es de un digito pasa a imprimirlo
487
488 0051 B60006      ldaa      celda2      ;si no carga el otro digito
489 0054 C630      ldab      #$30      ;se lo convierte a ASCII
490 0056 1B                      aba
491 0057 18A701      staa      1,y      ;se retiene el caracter
492
493 005A C601      undigit      ldab      #DI_END+1
494
495 005C 18A600      dosdig      ldaa      0,y
496 005F 2708      beq      endshow      ;se imprime el numero entero hasta ahora, ya sea
497 0061 BD0010      jsr      outchar      ;de una o dos cifras
498 0064 5A                      decb
499 0065 1808      iny
500 0067 20F3      bra      dosdig
501 0069 39      endshow      rts      ;vuelve a barrer
502
503
504
505 *****
506 *          SUBROUTINA ESPERE          *
507 *          *                         *
508 *      Le indica al jugador que no es *
509 *      su turno                       *
510 *          *                         *
511 *****
512
513 006A      espere      rseg      SUBR
514
515 006A C603      ldab      #DI_START      ;toma el valor de la primera posición del display
516 006C CE0004      ld      #4      ;inicia contador de posiciones del display
517 006F 8600      ldaa      #0      ;toma el valor 0 para borrar el display
518 0071 BD0010      jsr      outchar      ;llama a la subrutina para mandar al display
```



```
519 0074 5A          decb          ;decrementa la posición del display
520 0075 09          dex          ;decrementa el contador de posiciones
521 0076 26F7        bne      loops     ;si el contador no es 0 repite la secuencia
522
523 0078 18CE0016     ldy      #long      ;inicia contador de longitud de la frase
524 007C CE0001       ldx      #msge      ;apunta al primer caracter
525 007F BD008B       jsr      espere1    ;salta a la subrutina adecuada
526 0082 BD0017       jsr      delay      ;llama a la subrutina que genera el delay
527 0085 08          inx          ;incrementa el puntero de los caracteres
528 0086 1809        dey          ;decrementa el contador
529 0088 26F5        bne      repeat     ;si el contador no es 0, sigo imprimiendo
530 008A 39          rts          ;si el contador es 0, termino
531
532 008B             espere1      rseg      SUBR
533
534 008B 3C          pshx
535 008C 183C        pshy
536 008E 18CE0004     ldy      #4          ;inicia contador de caracteres a mostrar
537 0092 C603        ldab      #DI_START    ;toma el valor de la última posición del display
538 0094 A600        ldaa      0,x         ;toma el un caracter de la frase
539 0096 BD0010       jsr      outchar     ;llama a la subrutina para mandar al display
540 0099 1809        dey          ;decrementa contador de caracteres
541 009B 2705        beq      ter          ;si el contador es 0, termina la subrutina
542 009D 08          inx          ;si no es 0, incrementa el puntero de los caracteres
543 009E 5A          decb          ;decrementa la posición del display
544 009F 7E0094       jmp      loops2      ;vuelve a repetir la secuencia
545
546 00A2 1838        ter          ;retorna de la subrutina
547 00A4 38          pulx
548 00A5 39          rts
549
550
551
552 *****
553 *          SUBROUTINA VICTORIA          *
554 *                                          *
555 *      Le indica al jugador que no es    *
556 *          su turno                      *
557 *                                          *
558 *****
559
560 00A6             victoria      rseg      SUBR
561
562 00A6 C603        ldab      #DI_START    ;toma el valor de la primera posición del display
563 00A8 CE0004     ldx      #4          ;inicia contador de posiciones del display
564 00AB 8600        ldaa      #0         ;toma el valor 0 para borrar el display
565 00AD BD0010       jsr      outchar     ;llama a la subrutina para mandar al display
566 00B0 5A          decb          ;decrementa la posición del display
567 00B1 09          dex          ;decrementa el contador de posiciones
568 00B2 26F7        bne      lopes      ;si el contador no es 0 repite la secuencia
569
570 00B4 18CE000F     ldy      #lng        ;inicia contador de longitud de la frase
571 00B8 CE0018     ldx      #mge        ;apunta al primer caracter
572 00BB BD00C7       jsr      victoria1    ;salta a la subrutina adecuada
573 00BE BD0017       jsr      delay      ;llama a la subrutina que genera el delay
574 00C1 08          inx          ;incrementa el puntero de los caracteres
575 00C2 1809        dey          ;decrementa el contador
576 00C4 26F5        bne      rpeat      ;si el contador no es 0, sigo imprimiendo
577 00C6 39          rts          ;si el contador es 0, termino
578
579 00C7             victoria1      rseg      SUBR
```



```
580
581 00C7 3C          pshx
582 00C8 183C        pshy
583 00CA 18CE0004    ldy      #4          ;inicia contador de caracteres a mostrar
584 00CE C603        ldab     #DI_START   ;toma el valor de la última posición del display
585 00D0 A600      lopes2  ldaa     0,x      ;toma el un caracter de la frase
586 00D2 BD0010      jsr      outchar     ;llama a la subrutina para mandar al display
587 00D5 1809        dey      ;decrementa contador de caracteres
588 00D7 2705        beq      te          ;si el contador es 0, termina la subrutina
589 00D9 08          inx      ;si no es 0, incrementa el puntero de los caracteres
590 00DA 5A          decb     ;decrementa la posición del display
591 00DB 7E00D0      jmp      lopes2      ;vuelve a repetir la secuencia
592
593 00DE 1838      te          puly
594 00E0 38        pulx
595 00E1 39        rts          ;retorna de la subrutina
596
597
598
599 *****
600 *          SUBROUTINA SELEC          *
601 *          *                          *
602 * Le indica al jugador que seleccione *
603 *          mapas                    *
604 *          *                          *
605 *****
606
607 00E2      selec      rseg      SUBR
608
609 00E2 C603      ldab     #DI_START   ;toma el valor de la primera posición del display
610 00E4 CE0004    ldx      #4          ;inicia contador de posiciones del display
611 00E7 8600      loopess  ldaa     #0      ;toma el valor 0 para borrar el display
612 00E9 BD0010      jsr      outchar     ;llama a la subrutina para mandar al display
613 00EC 5A        decb     ;decrementa la posición del display
614 00ED 09        dex      ;decrementa el contador de posiciones
615 00EE 26F7      bne      loopess      ;si el contador no es 0 repite la secuencia
616
617 00F0 18CE0016   ldy      #lg          ;inicia contador de longitud de la frase
618 00F4 CE0028     ldx      #mg          ;apunta al primer caracter
619 00F7 BD0103     jsr      selec1       ;salta a la subrutina adecuada
620 00FA BD0017     jsr      delay       ;llama a la subrutina que genera el delay
621 00FD 08         inx      ;incrementa el puntero de los caracteres
622 00FE 1809        dey      ;decrementa el contador
623 0100 26F5      bne      repeatt      ;si el contador no es 0, sigo imprimiendo
624 0102 39        rts          ;si el contador es 0, termino
625
626 0103      selec1     rseg      SUBR
627
628 0103 3C          pshx
629 0104 183C        pshy
630 0106 18CE0004    ldy      #4          ;inicia contador de caracteres a mostrar
631 010A C603        ldab     #DI_START   ;toma el valor de la última posición del display
632 010C A600      loopess2  ldaa     0,x      ;toma el un caracter de la frase
633 010E BD0010      jsr      outchar     ;llama a la subrutina para mandar al display
634 0111 1809        dey      ;decrementa contador de caracteres
635 0113 2705        beq      tr          ;si el contador es 0, termina la subrutina
636 0115 08         inx      ;si no es 0, incrementa el puntero de los caracteres
637 0116 5A        decb     ;decrementa la posición del display
638 0117 7E010C     jmp      loopess2     ;vuelve a repetir la secuencia
639
640 011A 1838      tr          puly
```



```
641 011C 38          pulx
642 011D 39          rts          ;retorna de la subrutina
643
644
645
646 *****
647 *          AREA DE VARIABLES          *
648 *                                          *
649 *  Aqui se guardan las posiciones de  *
650 *  memoria que seran usadas como    *
651 *          variables                  *
652 *                                          *
653 *****
654
655 0000          rseg  VARS
656
657 0000          counter      rmb  1          ;este contador se guarda la cantidad de caracteres
658
659 0001 00          fcb  0
660 0002          msg         rmb  2          ;esto se usa para retener los caracteres ASCII
661 0004 00          fcb  0
662
663 0005          celda1       rmb  1          ;estas posiciones se usan para retener las celdas
664 0006          celda2       rmb  1          ;seleccionadas por los jugadores
665
666 0007          flagp        rmb  1          ;este flag indica que jugador esta en turno
667
668 0008          winp1        rmb  1          ;indican cuanto falta para que gane cada jugador
669 0009          winp2        rmb  1
670
671 000A          flagmsel     rmb  1          ;se usan para la seleccion de mapas
672 000B          flagmap      rmb  1
673
674
675
676 *****
677 *          AREA DE CONSTANTES          *
678 *                                          *
679 *  Aqui se guardan las posiciones de  *
680 *  memoria que seran usadas como    *
681 *          variables                  *
682 *                                          *
683 *****
684
685 0000          rseg  CONST
686
687 0000 00          fcb  0
688 0001 00          msge      fcb  0          ;necesarios para el comienzo de la muestra de la frase
689 0002 00          fcb  0
690 0003 00          fcb  0
691 0004 45535045     fcc  'ESPERE SU TURNO';define la frase que se quiere mostrar
692 0008 52452053
693 000C 55205455
694 0010 524E4F
695
696 0013 00          fcb  0          ;terminador
697 0014 00          fcb  0          ;necesarios para el final de la muestra de la frase
698 0015 00          fcb  0
699 0016 00          fcb  0
700 0016          long      equ  *-msge          ;define la longitud de la frase con más parámetros
```



699	0017 00		fc	0	
700	0018 00	mge	fc	0	;necesarios para el comienzo de la muestra de la frase
701	0019 00		fc	0	
702	001A 00		fc	0	
703	001B 56494354		fcc	'VICTORIA'	;define la frase que se quiere mostrar
	001F 4F524941				
704	0023 00		fc	0	;terminador
705	0024 00		fc	0	;necesarios para el final de la muestra de la frase
706	0025 00		fc	0	
707	0026 00		fc	0	
708	000F	lng	equ	*-mge	;define la longitud de la frase con más parámetros
709					
710	0027 00		fc	0	
711	0028 00	mg	fc	0	;necesarios para el comienzo de la muestra de la frase
712	0029 00		fc	0	
713	002A 00		fc	0	
714	002B 53454C45		fcc	'SELECCIONE MAPA'	;define la frase que se quiere mostrar
	002F 4343494F				
	0033 4E45204D				
	0037 415041				
715	003A 00		fc	0	;terminador
716	003B 00		fc	0	;necesarios para el final de la muestra de la frase
717	003C 00		fc	0	
718	003D 00		fc	0	
719	0016	lg	equ	*-mg	;define la longitud de la frase con más parámetros
720					
721					
722	003E		rseg	CONST	
723					
724	003E 03030383	MAP1	fc	AGUA,AGUA,AGUA,CARG,AGUA,CARG	
	0042 0383				
725	0044 03030383		fc	AGUA,AGUA,AGUA,CARG,AGUA,CARG	
	0048 0383				
726	004A 03430303		fc	AGUA,BARC,AGUA,AGUA,AGUA,AGUA	
	004E 0303				
727	0050 03838303		fc	AGUA,CARG,CARG,AGUA,AGUA,BARC	
	0054 0343				
728	0056 03030303		fc	AGUA,AGUA,AGUA,AGUA,AGUA,AGUA	
	005A 0303				
729	005C 03430303		fc	AGUA,BARC,AGUA,AGUA,AGUA,AGUA	
	0060 0303				
730					
731	0062 83030303	MAP2	fc	CARG,AGUA,AGUA,AGUA,CARG,CARG	
	0066 8383				
732	0068 83030343		fc	CARG,AGUA,AGUA,BARC,AGUA,AGUA	
	006C 0303				
733	006E 03030303		fc	AGUA,AGUA,AGUA,AGUA,AGUA,AGUA	
	0072 0303				
734	0074 43030303		fc	BARC,AGUA,AGUA,AGUA,BARC,AGUA	
	0078 4303				
735	007A 03038383		fc	AGUA,AGUA,CARG,CARG,AGUA,AGUA	
	007E 0303				
736	0080 03030303		fc	AGUA,AGUA,AGUA,AGUA,AGUA,AGUA	
	0084 0303				
737					
738	0086 03030343	MAP3	fc	AGUA,AGUA,AGUA,BARC,AGUA,AGUA	
	008A 0303				
739	008C 83830303		fc	CARG,CARG,AGUA,AGUA,AGUA,AGUA	
	0090 0303				
740	0092 03038303		fc	AGUA,AGUA,CARG,AGUA,AGUA,AGUA	
	0096 0303				



Introducción a la Computación 22.06
Trabajo Práctico Final

741	0098 03038303		fcB	AGUA, AGUA, CARG, AGUA, CARG, CARG
	009C 8383			
742	009E 03430303		fcB	AGUA, BARC, AGUA, AGUA, AGUA, AGUA
	00A2 0303			
743	00A4 03030343		fcB	AGUA, AGUA, AGUA, BARC, AGUA, AGUA
	00A8 0303			
744				
745	00AA 43030303	MAP4	fcB	BARC, AGUA, AGUA, AGUA, AGUA, AGUA
	00AE 0303			
746	00B0 03034303		fcB	AGUA, AGUA, BARC, AGUA, AGUA, BARC
	00B4 0343			
747	00B6 03830303		fcB	AGUA, CARG, AGUA, AGUA, AGUA, AGUA
	00BA 0303			
748	00BC 03830383		fcB	AGUA, CARG, AGUA, CARG, CARG, AGUA
	00C0 8303			
749	00C2 03030303		fcB	AGUA, AGUA, AGUA, AGUA, AGUA, AGUA
	00C6 0303			
750	00C8 03838303		fcB	AGUA, CARG, CARG, AGUA, AGUA, AGUA
	00CC 0303			
751				
752	00CE 03830303	MAP5	fcB	AGUA, CARG, AGUA, AGUA, AGUA, AGUA
	00D2 0303			
753	00D4 03830383		fcB	AGUA, CARG, AGUA, CARG, CARG, AGUA
	00D8 8303			
754	00DA 03030303		fcB	AGUA, AGUA, AGUA, AGUA, AGUA, AGUA
	00DE 0303			
755	00E0 03030303		fcB	AGUA, AGUA, AGUA, AGUA, AGUA, BARC
	00E4 0343			
756	00E6 43030303		fcB	BARC, AGUA, AGUA, AGUA, AGUA, AGUA
	00EA 0303			
757	00EC 03034303		fcB	AGUA, AGUA, BARC, AGUA, CARG, CARG
	00F0 8383			
758				
759	00F2 03030303	MAP6	fcB	AGUA, AGUA, AGUA, AGUA, BARC, AGUA
	00F6 4303			
760	00F8 43038303		fcB	BARC, AGUA, CARG, AGUA, AGUA, AGUA
	00FC 0303			
761	00FE 03038303		fcB	AGUA, AGUA, CARG, AGUA, AGUA, CARG
	0102 0383			
762	0104 03030303		fcB	AGUA, AGUA, AGUA, AGUA, AGUA, CARG
	0108 0383			
763	010A 83030343		fcB	CARG, AGUA, AGUA, BARC, AGUA, AGUA
	010E 0303			
764	0110 83030303		fcB	CARG, AGUA, AGUA, AGUA, AGUA, AGUA
	0114 0303			
765				
766	0116 83830303	MAP7	fcB	CARG, CARG, AGUA, AGUA, AGUA, AGUA
	011A 0303			
767	011C 03030303		fcB	AGUA, AGUA, AGUA, AGUA, AGUA, AGUA
	0120 0303			
768	0122 03038383		fcB	AGUA, AGUA, CARG, CARG, AGUA, BARC
	0126 0343			
769	0128 43030303		fcB	BARC, AGUA, AGUA, AGUA, AGUA, AGUA
	012C 0303			
770	012E 03038383		fcB	AGUA, AGUA, CARG, CARG, AGUA, AGUA
	0132 0303			
771	0134 03030303		fcB	AGUA, AGUA, AGUA, AGUA, BARC, AGUA
	0138 4303			
772				
773				



```
774
775 *****
776 *          AREA DEL STACK          *
777 *
778 *   Aqui se guardan las posiciones de *
779 *   memoria que seran usadas como  *
780 *           stack                    *
781 *
782 *****
783
784 0000                      rseg      STACK
785
786 0000          stack      rmb       100
787 0063          stackP     EQU      *-1
788
789
790
791 0064                      END
```

```
Errors:  None          #####
Bytes:   1295          # tbn #
CRC:     2D1B          #####
```

FIREC.LST

```
#####
#
#   Micro Series 6801 Assembler V2.00/DOS          10/Nov/20  15:01:59  #
#
#   Source   =   firec.msa                      #
#   List     =   firec.lst                      #
#   Object   =   firec.r07                      #
#   Options  =
#
#
#                                     (c) Copyright IAR Systems 1990 #
#####
```

```
1 0000          p68h11
2
5
6 0000          extern  flagp,winp1,winp2,EL_SIZE,AR_SIZE,MAPA1,MAPA2,AGUA,BARC,CARG
7 0000          public  dofire,loop1,BM
8
9
10 *****
11 *          AREA DE MACROS          *
12 *
13 *   Aqui se definen las macros que *
14 *   seran utilizadas a lo largo   *
15 *           del programa           *
16 *
17 *****
18
19 0000          MACRO    %IMPR          ;macro que se encacarga de imprimir la matriz en el bitmap
20
21                      ldaa    #\0          ;inicia contador de cantidad de casillas
22          again\@    ldab    0,x          ;toma una casilla de la matriz
23                      stab    0,y          ;toma una posición del bitmap
24                      inx          ;apunta a la casilla siguiente
```



```
25          iny          ;apunta a la posición del bitmap siguiente
26          deca         ;decrementa el contador de casillas
27          bne    again\< ;si el contador no es cero, repite el proceso
28 0000          ENDMAC
29
30 0000          MACRO    %ANALI          ;analiza los que pasa en las posiciones alrededor del carguero
31          ldy    guardo          ;toma la posición de la casilla
32          ldab   #\0             ;toma el valor absoluto de desplazamiento
33          aby          ;le suma el desplazamiento
34          ldab   0,y             ;analiza si la casilla tiene un carguero hundido
35 0000          ENDMAC
36
37 0000          MACRO    %ANALI2         ;analiza los que sucede en las posiciones alrededor del carguero
38          ldab   #\0             ;inicia contador de posiciones a moverse hacia atrás
39          ldy    guardo          ;toma la posición de la casilla
40          repe\@  dey            ;el puntero apunta a una posición anterior
41          decb          ;decrementa el contador
42          bne    repe\<          ;si el contador no es cero, repite el proceso
43          ldab   0,y             ;analiza si la casilla tiene un carguero hundido
44 0000          ENDMAC
45
46
47          *****
48          *                AREA DE EQU          *
49          *                *
50          *  Se definen labels que se utilizaran *
51          *                en la subrutina      *
52          *                *
53          *****
54
55 0006          N      EQU    6          ;filas
56 0006          M      EQU    6          ;columnas
57 0024          MATRIZ EQU    N*M
58 0030          ROJO   EQU    00110000b    ;valor del color rojo
59 000C          VERDE  EQU    00001100b    ;valor del color verde
60 003C          AMA    EQU    00111100b    ;valor del color amarillo
61 00BC          TOCAC  EQU    10111100b    ;valor que indica que un carguero ya está tocado
62 00C0          MASC   EQU    11000000b    ;máscara utilizada para eliminar los colores
63 5F00          BM     EQU    $5F00        ;comienzo del bitmap
64 0005          five   EQU    5
65 0006          six    EQU    6
66 0001          one    EQU    1
67
68
```



```
69
70 *****
71 *           SUBROUTINA DOFIRE           *
72 *                                     *
73 * Se encarga de analizar el indice que *
74 * se disparo y dependiendo de lo que  *
75 * encontraba en la casilla manda      *
76 * el resultado al bitmap              *
77 *                                     *
78 *****
79
80 0000          rseg    SUBR
81
82
83 *****
84 *           MODULO 1                   *
85 *                                     *
86 * Se encarga de imprimir la matriz    *
87 * en el bitmap                        *
88 *                                     *
89 *****
90
91 0000 B60000    dofire  ldaa    flagp          ;toma el valor de la constante que indica el jugador
92 0003 8101      cmpa    #1                    ;si el valor es uno, está jugando el primer jugador
93 0005 2706      beq     pj                    ;
94 0007 CE0000    ldx     #MAPA2                ;si no es uno, toma el valor del inicio de la matriz de 2° jugador
95 000A 7E0010    jmp     impre                 ;
96 000D CE0000    pj      ldx     #MAPA1          ;toma el valor del comienzo de la matriz del primer jugador
97 0010 18CE5F00 impre  ldy     #BM              ;toma el valor del comienzo del bitmap
98 0014          %IMPR   MATRIZ                ;manda la matriz al bitmap
99
100
101 *****
102 *           MODULO 2                   *
103 *                                     *
104 * Se encarga de tomar el indice        *
105 * disparado por stack y analizar que   *
106 * se encuentra en la casilla          *
107 *                                     *
108 *****
109
110 0021 1830      tsy          ;crea el frame pointer
111 0023 B60000    ldaa    flagp          ;toma el valor de la constante que indica el jugador
112 0026 8101      cmpa    #1                    ;
113 0028 2703      beq     pj1                ;si es uno, está jugando el primer jugador
114 002A 7E0033    jmp     sj1                ;si no es uno, está jugando el segundo jugador
115
116 002D CE0000    pj1      ldx     #MAPA1          ;toma el valor del comienzo de la matriz del primer jugador
117 0030 7E0036    jmp     indi                ;
118 0033 CE0000    sj1      ldx     #MAPA2          ;toma el valor del comienzo de la matriz del segundo jugador
119 0036 18A603    indi     ldaa    3,y          ;toma el índice de fila ingresado por stack
120 0039 C600      ldab     #AR_SIZE            ;toma el tamaño de las filas
121 003B 3D        mul
122 003C 3A        abx
123 003D 18A602    ldaa    2,y          ;toma el índice de columna ingresado por stack
124 0040 C600      ldab     #EL_SIZE            ;toma el tamaño de las columnas
125 0042 3D        mul
126 0043 3A        abx          ;obtiene la posición correspondiente a la casilla ingresada
127 0044 A600      ldaa    0,x          ;toma el valor que hay en la casilla
128 0046 8100      cmpa    #AGUA          ;analiza si hay agua en la casilla
```



```
129 0048 270B      beq      ag
130 004A 8100      cmpa     #BARC      ;analiza si hay barco en la casilla
131 004C 2711      beq      bc
132 004E 8100      cmpa     #CARG      ;analiza si hay un carguero en la casilla
133 0050 2727      beq      car
134 0052 7E0152    jmp      loop1
135
136
137 *****
138 *              MODULO DE AGUA          *
139 *                                          *
140 *****
141
142 0055 84C0      ag      anda    #MASC      ;si es agua la casilla, saca el color azul del byte
143 0057 C60C      ldab     #VERDE     ;pasa a verde el byte
144 0059 1B        aba
145 005A A700      staa     0,x          ;pasa el byte a verde a la matriz
146 005C 7E0152    jmp      loop1     ;salta al la impresión de la matriz al bitmap
147
148
149 *****
150 *              MODULO DE BARCO         *
151 *                                          *
152 *****
153
154 005F 84C0      bc      anda    #MASC      ;si es barco la casilla, sacar el color azul del byte
155 0061 C630      ldab     #ROJO      ;pasa a rojo el byte
156 0063 1B        aba
157 0064 A700      staa     0,x          ;pasa el byte a rojo a la matriz
158 0066 F60000    ldab     flagp      ;toma el valor de la constante que indica el jugador
159 0069 C101      cmpb     #1          ;si es uno, está jugando el primer jugador
160 006B 2706      beq      cp1s
161 006D 7A0000    cp2s     dec      winp2      ;decrementa el contador de barcos hundidos del segundo jugador
162 0070 7E0152    jmp      loop1
163 0073 7A0000    cp1s     dec      winp1      ;decrementa el contador de barcos hundidos del primer jugador
164 0076 7E0152    jmp      loop1     ;salta a la impresión de la matriz al bitmap
165
166
167 *****
168 *              MODULO DE CARGUERO       *
169 *                                          *
170 *****
171
172 0079 84C0      car      anda    #MASC      ;si es carguero la casilla, saca el color azul del byte
173 007B FF0000    stx      guardo     ;guarda la posición de la casilla
174 007E 1830      tsy
175 0080 18E603    ldab     3,y          ;analiza si la fila es la primera
176 0083 C100      cmpb     #0
177 0085 2712      beq      fil0      ;la fila es la primera, analiza la columna
178 0087 C105      cmpb     #five     ;analiza si la fila es la última
179 0089 271C      beq      fil5      ;la fila es la última, analiza la columna
180 008B 18E602    ldab     2,y          ;la fila no es ni la primera ni la última
181 008E C100      cmpb     #0          ;analiza si la columna es la primera
182 0090 2764      beq      col0      ;la columna es la primera
183 0092 C105      cmpb     #five     ;analiza si la columna es la última
184 0094 272A      beq      col5      ;la columna es la última
185 0096 7E00B1    jmp      red          ;no está en las primeras ni últimas filas y columnas
186
187
188 0099 18E602    fil0     ldab     2,y          ;analiza si la columna es la primera
189 009C C100      cmpb     #0
```



```
190 009E 2711      beq    red      ;la fila y la column son las primeras
191 00A0 C105      cmpb   #five   ;analiza si la columna es la última
192 00A2 271C      beq    col5    ;la fila es la primera y la columna la última
193 00A4 7E00B1    jmp     red      ;la fila es la primera,la Columna no es ni la primera ni la última
194
195 00A7 18E602    fil5   ldab    2,y      ;analiza si la columna es la primera
196 00AA C100      cmpb   #0       ;
197 00AC 2748      beq    col0    ;la fila es la última, la columna la primera
198 00AE 7E00B1    jmp     red      ;
199
200 00B1          red     %ANALI one      ;analiza lo que pasa en la posición a la derecha
201 00BC C1BC      cmpb   #TOCAC
202 00BE 2771      beq    red1     ;si es un carguero ya tocado, pone en rojo las dos casillas
203
204 00C0          col5    %ANALI six      ;analiza lo que pasa en la posición por debajo de la casilla
205 00CB C1BC      cmpb   #TOCAC
206 00CD 2762      beq    red1     ;si es un carguero ya tocado, pone en rojo las dos casillas
207
208 00CF          %ANALI2 six      ;analiza lo que pasa en posición por arriba de la casilla
209 00DD C1BC      cmpb   #TOCAC
210 00DF 2750      beq    red1     ;si es un carguero ya tocado, pone en rojo las dos casilla
211
212 00E1          %ANALI2 one      ;analiza lo que pasa en la posición a la izquierda
213 00EF C1BC      cmpb   #TOCAC
214 00F1 273E      beq    red1     ;si es un carguero ya tocado, pone en rojo las dos casillas
215 00F3 7E0129    jmp     amari    ;no tiene cargueros tocados alrededor
216
217 00F6          col0    %ANALI one      ;analiza lo que pasa en la posición a la derecha
218 0101 C1BC      cmpb   #TOCAC
219 0103 272C      beq    red1     ;si es un carguero ya tocado, pone en rojo las dos casillas
220
221 0105          %ANALI six      ;analiza lo que pasa en la posición por debajo de la casilla
222 0110 C1BC      cmpb   #TOCAC
223 0112 271D      beq    red1     ;si es un carguero ya tocado, pone en rojo las dos casillas
224
225 0114          %ANALI2 six      ;analiza lo que pasa en posición por arriba de la casilla
226 0122 C1BC      cmpb   #TOCAC
227 0124 270B      beq    red1     ;si es un carguero ya tocado, pone en rojo las dos casilla
228 0126 7E0129    jmp     amari    ;no tiene cargueros tocados alrededor
229
230
231 0129 C63C      amari  ldab    #AMA     ;si no tiene alrededor cargueros tocados,pone casilla en amarillo
232 012B 1B        aba
233 012C A700      staa   0,x      ;y la guarda en la matriz
234 012E 7E0152    jmp     loop1    ;salta a la impresión de la matriz al bitmap
235
236
237 0131 C630      red1   ldab    #ROJO    ;pone en rojo la casilla disparada
238 0133 1B        aba
239 0134 A700      staa   0,x      ;y la guarda en la matriz
240 0136 18A600    ldaa   0,y      ;toma el valor de la casilla ya tocada
241 0139 84C0      anda   #MASC     ;aplica máscara para eliminar los colores que ya tenía
242 013B 1B        aba          ;pone la casilla en rojo
243 013C 18A700    staa   0,y      ;y la guarda en la matriz
244 013F F60000    ldab   flagp    ;toma el valor de la constante que indica el jugador
245 0142 C101      cmpb   #1       ;si es uno, está jugando el primer jugador
246 0144 2706      beq    cp1
247 0146 7A0000    cp2    dec     winp2    ;decrementa el contador de barcos hundidos del segundo jugador
248 0149 7E0152    jmp     loop1    ;salta a la impresión de la matriz al bitmap
249 014C 7A0000    cp1    dec     winp1    ;decrementa el contador de barcos hundidos del primer jugador
250 014F 7E0152    jmp     loop1    ;salta a la impresión de la matriz al bitmap
```



```
251
252
253
254 *****
255 *                MODULO 3                *
256 *                                          *
257 *   Se encarga de imprimir la matriz   *
258 *   modificada en el bitmap           *
259 *                                          *
260 *****
261
262 0152 B60000  loop1  ldaa  flagp          ;toma el valor de la constante que indica el jugador
263 0155 8101    cmpa  #1                  ;si es uno, está jugando el primer jugador
264 0157 2706    beq   pj2
265 0159 CE0000  ldx   #MAPA2              ;toma el valor de la matriz del segundo jugador
266 015C 7E0162  jmp   impre2
267
268 015F CE0000  pj2   ldx   #MAPA1          ;toma el valor de la matriz del primer jugador
269 0162 18CE5F00 impre2 ldy   #BM
270 0166        %IMPR MATRIZ              ;imprime la matriz al bitmap correspondiente al jugador
271 0173 39      rts
272
273
274 *****
275 *                AREA DE VARIABLES        *
276 *                                          *
277 *   Aqui se guardan las posiciones de   *
278 *   memoria que seran usadas como     *
279 *   variables                          *
280 *                                          *
281 *****
282
283 0000          rseg  VARS
284
285 0000  guardo  rmb   2
286
287
288
289 0002          END

Errors:  None      #####
Bytes:   372      # firec #
CRC:     565C     #####
```



WINNER.LST

```
#####
#
#   Micro Series 6801 Assembler V2.00/DOS           10/Nov/20  15:02:16   #
#
#   Source   =   winner.msa                               #
#   List     =   winner.lst                               #
#   Object   =   winner.r07                               #
#   Options  =                                           #
#
#                                           (c) Copyright IAR Systems 1990 #
#####

1  0000                p68H11
2
3  0000                extern  flagp,BM
4  0000                public  wins
5
6          *****
7          *          DEFINICIONES                                *
8          *                                                    *
9          *                                                    *
10         *                                                    *
11         *                                                    *
12         *                                                    *
13         *                                                    *
14         *                                                    *
15         *                                                    *
16         *****
17
18  5EFF                CONTROL EQU    $5EFF                ;Control de modo de la matriz
19  0006                COL    EQU    $6                    ;Cantidad de columnas
20  0006                FIL    EQU    $6                    ;Cantidad de filas
21  0024                SIZE    EQU    COL*FIL
22
23  0000                RGB    EQU    0                    ;\
24  0001                GRB    EQU    1                    ;\=> Defino constantes para modo de lectura
25
26  0028                YELLOW EQU    $28                  ;\
27  0030                RED    EQU    $30                  ;\
28  0002                BLUE   EQU    $02                  ;\=> Máscaras para cada color en RGB, filtro tenue
29
30  0001                win    EQU    1                    ;Constante para verificar quién ganó
31
32          *****
33          *          MAIN PROGRAM                                *
34          *          Esta sección analiza qué jugador ganó el juego y muestra *
35          *          el mensaje de victoria acorde.                *
36          *****
37
38  0000                rseg    SUBR
39
40  0000 8600            wins    ldaa    #RGB                ;\
41  0002 B75EFF          staa    CONTROL                    ;\=> Defino modo de lectura
42  0005 B60000          ldaa    flagp                      ;\
43  0008 8101            cmpa    #win                       ;\
44  000A 2706            beq     player1                    ;\=> Si ganó el jugador 1, salta a player 1
45
```



```
46 000C CE0024    player2 ldx    #COLOR2    ;Si ganó el jugador 2, carga el dibujo de victoria del jugador 2
47 000F 7E0015          jmp    mostrar
48
49 0012 CE0000    player1 ldx    #COLOR1    ;Carga el dibujo de victoria del jugador 1
50
51 0015 18CE0000  mostrar ldy    #BM        ;Carga el inicio de la matriz (bitmap)
52 0019 C624          ldab   #SIZE       ;Establece contador para copiar y pegar cada byte en el bitmap
53 001B A600          lop     ldaa   0,X    ;\
54 001D 18A700          staa   0,Y    ;\
55 0020 08            inx          ;\
56 0021 1808          iny          ;\
57 0023 5A            decb         ;\
58 0024 26F5          bne     lop      ;\=> Copia y pega cada color en el bitmap
59
60 0026 39            rts
61
62                *****
63                *          SECCION DE MATRICES          *
64                *                                          *
65                *                                          *
66                *                                          *
67                *                                          *
68                *****
69
70 0000            rseg   CONST
71
72 0000            COLOR1 EQU    *
73 0000 28022828    FCB     YELLOW,BLUE,YELLOW,YELLOW,BLUE,YELLOW
74 0004 0228          FCB     YELLOW,YELLOW,BLUE,RED,YELLOW,YELLOW
75 0006 28280230    FCB     YELLOW,BLUE,RED,RED,BLUE,YELLOW
76 000A 2828          FCB     YELLOW,BLUE,BLUE,RED,BLUE,YELLOW
77 0012 28020230    FCB     YELLOW,BLUE,BLUE,RED,BLUE,YELLOW
78 0016 0228          FCB     YELLOW,BLUE,BLUE,RED,BLUE,YELLOW
79 0018 28020230    FCB     YELLOW,BLUE,BLUE,RED,BLUE,YELLOW
80 001C 0228          FCB     YELLOW,BLUE,BLUE,RED,BLUE,YELLOW
81 0022 28020230    FCB     YELLOW,BLUE,BLUE,RED,BLUE,YELLOW
82 0026 0228
83
84 0024            rseg   CONST
85
86 0024            COLOR2 EQU    *
87 0024 28022828    FCB     YELLOW,BLUE,YELLOW,YELLOW,BLUE,YELLOW
88 0028 0228          FCB     YELLOW,YELLOW,RED,RED,YELLOW,YELLOW
89 002A 28280303    FCB     YELLOW,RED,BLUE,BLUE,RED,YELLOW
90 002E 2828          FCB     YELLOW,BLUE,BLUE,RED,BLUE,YELLOW
91 0030 28300202    FCB     YELLOW,BLUE,RED,BLUE,BLUE,YELLOW
92 0034 3028          FCB     YELLOW,RED,RED,RED,RED,YELLOW
93 0036 28020230    FCB     YELLOW,BLUE,BLUE,RED,BLUE,YELLOW
94 003A 0228
95 003C 28023002    FCB     YELLOW,BLUE,RED,BLUE,BLUE,YELLOW
96 0040 0228
97 0042 28303030    FCB     YELLOW,RED,RED,RED,RED,YELLOW
98 0046 3028
99
100 0048            END

Errors:  None          #####
Bytes:   111          # winner #
CRC:     CA34          #####
```




RANDOM.LST

```
#####
#
#   Micro Series 6801 Assembler V2.00/DOS           10/Nov/20   20:42:12   #
#
#   Source   =   random.msa                       #
#   List     =   random.lst                       #
#   Object   =   random.r07                       #
#   Options  =                                     #
#
#                                           (c) Copyright IAR Systems 1990 #
#####

1 0000                p68h11
2
5
6 0000                public  EL_SIZE,AR_SIZE,MAPA1,MAPA2,ran
7 0000                extern  flagmap,MAP1,MAP2,MAP3,MAP4,MAP5,MAP6,MAP7
8
9 0006                FIL    EQU    6                ;es el valor de las columnas
10 0006               COL    EQU    6                ;es el valor de las filas
11 0001               EL_SIZE EQU    1
12 0006               AR_SIZE EQU    EL_SIZE*COL
13 0011               VALOR  EQU    17               ;es una constante usada para usos diversos
14 0006               FIJOP  EQU    6                ;valor usado para contadores
15 0005               COLUM  EQU    5                ;
16 0024               COUNTW EQU    36               ;contador de espacios de cada matriz
17 0002               COUNT  EQU    2                ;valor usado para contador de suboperaciones
18 0000               REFER  EQU    0                ;valor usado para las referencias de la matriz
19 0001               OP1    EQU    1                ;aquí se colocan como constantes los
20 0002               OP2    EQU    2                ;valores numéricos de cada tecla usados para
21 0003               OP3    EQU    3                ;seleccionar los pares de mapas ya elegidos
22 0004               OP4    EQU    4
23 0005               OP5    EQU    5                ;previamente con el teclado
24 0006               OP6    EQU    6
25 0007               OP7    EQU    7
26
27 0000               MACRO   %FIJO
28
29                   ldaa     flagmap                ;se lee el flag en el cual se guarda un valor dependiendo
30                   ldab     #\0                    ;de la tecla que se pulsó y tomamos el valor de cada operacion
31                   sba      ;y se compara con el flag para ver si es una operacion a realizar
32
33 0000               ENDMAC
34
35 0000               MACRO   %PX
36
37                   ldab     #COUNTW                ;inicia un contador para la primer matriz del jugador P1
38                   stab     Conter
39                   ldx      #\0                    ;nos paramos en la matriz a usar
40                   lopi1\@ ldaa     0,y              ;se comienza en la primer posicion del mapa ya pre hecho
41                   iny      ;avanzamos un espacio para luego copiar otro elemento
42                   staa     0,x                    ;se escribe ese lugar del mapa y se va
43                   inx      ;copiando así cada lugar en
44                   dec      Conter                  ;ambos mapas para los 2 jugadores
45                   beq      ini2\                  ;si se terminó el contador va a la otra operacion
46                   jmp      lopi1\                  ;y sino sigue colocando los elementos
```



```
47      ini2\   ldab    #COUNTW    ;comienza el conteo para el jugador numero 2
48      stab    Conter    ;se vuelve a cargar el contador
49      ldx     #\1      ;no paramos en el mapa del jugador 2
50      lopi2\   ldaa     0,y      ;y comienzan a cargarse los elementos
51      iny
52      staa    0,x      ;hasta que termine el contador y termine la
53      inx      ;operacion de llenado
54      dec     Conter
55      bne     lopi2\
56 0000      ENDMAC
57
58 0000      rseg     SUBR
59 0000 C602   ran     ldab    #COUNT    ;se inicia el contador para poder comenzar el programa
60 0002 F7005C stab    dific
61 0005 CE0000 M1Ref   ldx     #M1      ;nos paramos en la primera parte de la referencia
62 0008 C606   LoopR   ldab    #FIJOP    ;incio el contador para colocar los 0
63 000A F7005A stab    countr
64 000D 8600   LoopRe  ldaa     #REFER
65 000F A700   staa    0,x      ;se llena los espacios de referenciado
66 0011 08     inx
67 0012 7A005A dec     countr
68 0015 2702   beq     M2Ref   ;si el contador llega a 0 se llena el otro espacio
69 0017 26F4   bne     LoopRe  ;sino vuelve a su espacio inicial
70 0019 7A005C M2Ref   dec     dific      ;se ve si no va al tercer referenciado,
71 001C 270B   beq     MF1Ref
72 001E CE0054 ldx     #M2      ;comienza el segundo referenciado llenado de ceros
73 0021 C606   ldab    #FIJOP    ;los 6 espacios en el medio de los 2 mapas (jugador
74 0023 F7005A stab    countr      ;1 y jugador 2)
75 0026 7E000D jmp     LoopRe
76
77 0029 CE002A MF1Ref   ldx     #MF1      ;se posiciona en las ultimas referencias de ceros
78 002C C606   ldab    #FIJOP    ;se carga el valor del contador
79 002E F7005A stab    countr
80 0031 8600   Loop2R  ldaa     #REFER
81 0033 A700   staa    0,x
82 0035 08     inx      ;comienza el llenado de ceros hasta terminar
83 0036 7A005A dec     countr      ; e ir a la lectura de flag de la tecla
84 0039 2703   beq     start
85 003B 7E0031 jmp     Loop2R
86
87
88
89 003E B60000 start    ldaa     flagmap    ;se lee el flag de la tecla seleccionada y
90
91 0041      opera   %FIJO  1      ;dependiendo de qué tecla de presionó va a hacer
92 0047 2634      bne     opera2    ;un llenado de embarcaciones diferentes
93
94 0049 18CE0000      ldy     #MAP1      ;ve si es el mapa correspondiente en cada caso, ya sea 1,2..
95 004D      %PX     MAPA1,MAPA2    ; en el caso de que sea alguna opcion
96 007A 7E01A9      jmp     Termi    ;realiza una operacion que consiste en llenar el Mapa del P1
97 007D      opera2 %FIJO  2      ;y luego el Mapa del P2
98 0083 2634      bne     opera3    ;logrando así una configuracion distinta paracada jugador
99
100 0085 18CE0000      ldy     #MAP2
101 0089      %PX     MAPA1,MAPA2
102 00B6 7E01A9      jmp     Termi
103
104 00B9      opera3 %FIJO  3
105 00BF 2634      bne     opera4
106
107 00C1 18CE0000      ldy     #MAP3
```



```
108 00C5          %PX  MAPA1,MAPA2
109 00F2 7E01A9   jmp   Termi
110
111 00F5          opera4 %FIJO 4
112 00FB 2634     bne   opera5
113
114 00FD 18CE0000   ldy   #MAP4
115 0101          %PX  MAPA1,MAPA2
116 012E 7E01A9   jmp   Termi
117
118 0131          opera5 %FIJO 5
119 0137 2634     bne   opera6
120
121 0139 18CE0000   ldy   #MAP5
122 013D          %PX  MAPA1,MAPA2
123 016A 7E01A9   jmp   Termi
124
125 016D          opera6 %FIJO 6
126 0173 2634     bne   Termi
127
128 0175 18CE0000   ldy   #MAP6
129 0179          %PX  MAPA1,MAPA2
130 01A6 7E01A9   jmp   Termi
131
132 01A9 39        Termi rts                ;termina y comienza la partida
133
134 0000          rseg  CONST
135 0000          M1    rmb  6                ;aquí estan los 6 espacios para referencia inicial
136 0006          MAPA1 rmb  36              ;aquí está el mapa del P1
137 002A          MF1   rmb  6                ;segundo espacio de referencia
138 0030          MAPA2 rmb  36              ;aquí esta el mapa del P2
139 0054          M2    rmb  6                ;tercer espacio de referencia
140 005A          countr rmb  1
141 005B          Conter rmb  1
142 005C          dific rmb  1
143
144 005D          END

Errors:  None      #####
Bytes:   426      # random #
CRC:     4963     #####
```

LI11.XCL

```
-ZCODE=2000          -! Code segment -!
-ZVARS=3000
-ZCONST=4000
-ZSUBR=4500
-ZSTACK=5500        -! Data segment -!
-c68hc11             -! Processor -!
-Fmotorola           -! Output Format -!
-x                   -! Cross Reference -!
-B                   -! Forced dump -!
```



LINKOUT.LST

```
#####  
#  
#      Micro Series Universal Linker V4.46C/DXT      10/Nov/120  22:47:42      #  
#  
#      Target CPU      = 68hc11      #  
#      List file       = linkout.lst  #  
#      Output file 1   = aout.a07     #  
#      Output format  = motorola      #  
#      Command line   = -f li11 (-ZCODE=2000 -ZVARS=3000 -ZCONST=4000      #  
#                      -ZSUBR=4500 -ZSTACK=5500 -c68hc11 -Fmotorola -x      #  
#                      -B) -A tbn firec winner random -l linkout.lst      #  
#  
#                      (c) Copyright IAR Systems 1993 #  
#####
```

```
*****  
*                                *  
*      CROSS REFERENCE          *  
*                                *  
*****
```

There are no program entries.

```
*****  
*                                *  
*      MODULE MAP               *  
*                                *  
*****
```

FILE NAME : tbn.r07

PROGRAM MODULE, NAME : tbn

ABSOLUTE ENTRIES	ADDRESS	REF BY MODULE
=====	=====	=====
AGUA	0003	firec
BARC	0043	firec
CARG	0083	firec

SEGMENTS IN THE MODULE

=====

CODE

Relative segment, address : 2000 - 22B4

SUBR

Relative segment, address : 4500 - 461D

VARs

Relative segment, address : 3000 - 300B

ENTRIES	ADDRESS	REF BY MODULE
flagp	3007	firec
		winner
winp1	3008	firec
winp2	3009	firec



flagmap	300B	random
---------	------	--------

CONST

Relative segment, address : 4000 - 4139

ENTRIES	ADDRESS	REF BY MODULE
MAP1	403E	random
MAP2	4062	random
MAP3	4086	random
MAP4	40AA	random
MAP5	40CE	random
MAP6	40F2	random
MAP7	4116	random

STACK

Relative segment, address : 5500 - 5563

FILE NAME : firec.r07

PROGRAM MODULE, NAME : firec

ABSOLUTE ENTRIES	ADDRESS	REF BY MODULE
=====	=====	=====
BM	5F00	winner

SEGMENTS IN THE MODULE

=====

SUBR

Relative segment, address : 461E - 4791

ENTRIES	ADDRESS	REF BY MODULE
dofire	461E	tbn
loop1	4770	tbn

VARS

Relative segment, address : 300C - 300D

FILE NAME : winner.r07

PROGRAM MODULE, NAME : winner

SEGMENTS IN THE MODULE

=====

SUBR

Relative segment, address : 4792 - 47B8

ENTRIES	ADDRESS	REF BY MODULE
wins	4792	tbn

CONST

Relative segment, address : 413A - 4181

FILE NAME : random.r07

PROGRAM MODULE, NAME : random

ABSOLUTE ENTRIES	ADDRESS	REF BY MODULE
=====	=====	=====
EL_SIZE	0001	firec
AR_SIZE	0006	firec



SEGMENTS IN THE MODULE

=====

SUBR

Relative segment, address : 47B9 - 4962

ENTRIES	ADDRESS	REF BY MODULE
ran	47B9	tbn

CONST

Relative segment, address : 4182 - 41DE

ENTRIES	ADDRESS	REF BY MODULE
MAPA1	4188	firec
MAPA2	41B2	firec

*
* SEGMENTS IN DUMP ORDER *
*

SEGMENT	START ADDRESS	END ADDRESS	TYPE	ORG	P/N	ALIGN
=====	=====	=====	=====	=====	=====	=====
CODE	2000	- 22B4	rel	stc	pos	0
VARs	3000	- 300D	rel	stc	pos	0
CONST	4000	- 41DE	rel	stc	pos	0
SUBR	4500	- 4962	rel	stc	pos	0
STACK	5500	- 5563	rel	stc	pos	0

*
* END OF CROSS REFERENCE *
*

Errors: none
Warnings: none